FastBPE: Benchmarking and Optimizing Tokenizer Spee

Research-style benchmark report

FastBPE benchmarks tokenizer throughput, latency, memory, batching behavior, and domain sensitivity across English, code, web, and technical text. In the current five-trial run on Windows-10-10.0.26200-SP0, tiktoken_cached was the strongest average MB/s performer at 5.253 MB/s. SentencePiece remained competitive and led the code domain at 5.134 MB/s. The cached Python tokenizer improved on the naive Python baseline by 0.58x in MB/s with higher memory usage. The exact-compatible tiktoken_cached path preserved token IDs exactly and now slightly outperformed native tiktoken (5.253 vs 5.167 MB/s).

Hypotheses

H1. Production native tokenizer libraries will outperform readable Python baselines on throughput and latency.

H2. Tokenizer rankings will change across English prose, code, noisy web text, and technical markdown-like text.

H3. Repeated-substring caching will improve the custom Python baseline while increasing memory usage.

H4. Batch encoding will favor tokenizers that expose efficient multi-document paths, but speed gains will not be uniform.

H5. Exact token-ID compatibility cannot be claimed unless the tokenizer intentionally matches the reference vocabulary and merge logic.

H6. Even if exactness is preserved, any speed gain from caching may be modest and may come with substantial memory overhead.

Methodology and Experimental Design

Environment. Platform: Windows-10-10.0.26200-SP0. Python: 3.11.9. Processor: Intel64 Family 6 Model 189 Stepping 1, GenuineIntel.

Tokenizers. tiktoken, tiktoken_cached, hf, sentencepiece, naive, cached.

Domains. code, english, technical, web.

Controls. Warmup before timing, separate cold-start measurement, identical document sets per run, five trials, and raw CSV/JSON output.

Datasets. This run used persisted fetched corpora in data/ rather than the in-memory homogeneous fallback.

Exactness. tiktoken is the reference tokenizer, and this run includes tiktoken_cached as an intentionally exact-compatible cached path.

Experiments

Experiment 1: Overall tokenizer speed, measured in MB/s, tokens/s, documents/s, and latency percentiles.

Experiment 2: Domain sensitivity across clean English, code, noisy web text, and technical text.

Experiment 3: Input-length scaling across the available length buckets in the current corpus.

Experiment 4: Batch versus single encoding, reported for tokenizers that support encode_batch.

Experiment 5: Exactness testing for the tiktoken_cached exact-compatible adapter against the tiktoken reference.

Experiment 6: Caching optimization, comparing the naive and cached Python prototypes.

Overall Results by Tokenizer

This table aggregates mean performance across all domains and trials.

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken_cached	5.253	1230720	0.2143	2233979.0
tiktoken	5.167	1211488	0.2233	120947.6
sentencepiece	3.058	1566243	0.3750	199494.0
hf	3.015	928416	0.3639	114754.0
naive	1.365	1225939	0.8172	207623.3
cached	0.788	711538	1.4070	1235830.0

Top Tokenizer-Domain Configurations

These are the strongest individual tokenizer-domain combinations by MB/s.

tokenizer	domain	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken_cached	technical	6.376	1290529	0.2313	2472672.0
tiktoken	technical	6.291	1273319	0.2344	35843.8
tiktoken	english	5.138	1267728	0.1387	46248.0
tiktoken_cached	code	5.134	1219410	0.4309	4470460.0
tiktoken_cached	english	4.985	1229895	0.1431	1456340.0
tiktoken	code	4.731	1123829	0.4682	370000.8
tiktoken_cached	web	4.517	1183047	0.0518	536444.0
tiktoken	web	4.509	1181076	0.0518	31697.6
hf	code	3.552	1040519	0.6233	320944.0
sentencepiece	english	3.315	1596033	0.2156	37904.0

Domain Sensitivity: MB/s by Domain

Throughput shifts materially by domain, which changes the ranking of libraries.

domain	cached	hf	naive	sentencepiece	tiktoken	tiktoken_cached
code	1.127	3.552	1.463	3.098	4.731	5.133743281579646
english	0.550	3.105	1.472	3.315	5.138	4.98460850642769
technical	0.778	3.051	1.408	3.084	6.291	6.376365956809692
web	0.696	2.353	1.118	2.736	4.509	4.516598645626086

Cold Start and Batch Results

Cold start is reported for all tokenizers. Batch metrics are available only for adapters with batch support.

tokenizer	metric	value	docs_per_s	tokens_per_s	batch_latency_ms
tiktoken	cold_start_ms	0.0965			
tiktoken_cached	cold_start_ms	0.0993			
naive	cold_start_ms	0.1262			
cached	cold_start_ms	0.2405			
hf	cold_start_ms	0.2733			
sentencepiece	cold_start_ms	0.3502			
tiktoken_cached			1135351.3	212204863	0.0085
naive			12501.2	7282778	1.1036
cached			10347.7	4763151	1.9666
hf			8641.2	2203452	1.1276
sentencepiece			6420.6	2962452	1.3784

Input-Length Scaling Summary

The current persisted corpus produced two dominant length buckets, so the length-scaling analysis is informative but not yet broad.

tokenizer	length_bucket	latency_ms	bytes	tokens
cached	1025-4096	2.0064	1840.1	1589.3
cached	257-1024	0.9831	591.5	485.8
cached	4097+	16.2059	19289.0	17268.0
cached	65-256	0.2082	135.7	124.3
hf	1025-4096	0.5348	1840.1	503.4
hf	257-1024	0.1902	591.5	175.0
hf	4097+	5.5885	19289.0	5243.0
hf	65-256	0.0639	135.7	56.6
naive	1025-4096	1.2213	1840.1	1589.3
naive	257-1024	0.3914	591.5	485.8
naive	4097+	12.9944	19289.0	17268.0
naive	65-256	0.1260	135.7	124.3
sentencepiece	1025-4096	0.5644	1840.1	851.6
sentencepiece	257-1024	0.1736	591.5	274.2
sentencepiece	4097+	6.1060	19289.0	8348.0
sentencepiece	65-256	0.0529	135.7	86.8
tiktoken	1025-4096	0.3295	1840.1	392.2
tiktoken	257-1024	0.1097	591.5	137.1
tiktoken	4097+	4.1729	19289.0	4750.0
tiktoken	65-256	0.0336	135.7	37.7
tiktoken_cached	1025-4096	0.3133	1840.1	392.2
tiktoken_cached	257-1024	0.1138	591.5	137.1
tiktoken_cached	4097+	3.7027	19289.0	4750.0
tiktoken_cached	65-256	0.0343	135.7	37.7

Caching Optimization Results

Repeated-substring caching improved throughput and latency relative to the naive Python baseline, with a memory cost.

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_byte	cache_hit_rate
cached	0.788	711538	1.4070	1235830.0	0.8804
naive	1.365	1225939	0.8172	207623.3	N/A

Exact-Compatible Cached Results

The exact-compatible cached adapter is evaluated separately because the key question is whether correctnes
can be preserved while optimizing.

tokenizer	domain	exact_match_rate
tiktoken_cached	code	1.0000
tiktoken_cached	english	1.0000
tiktoken_cached	technical	1.0000
tiktoken_cached	web	1.0000

Direct Comparison: tiktoken vs tiktoken_cached

This table isolates the most important comparison in the project: the native reference versus the exact-compatible cached variant.

tokenizer	domain	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes	exact_match_rate
tiktoken	code	4.731	1123829	0.4682	370000.8	N/A
tiktoken	english	5.138	1267728	0.1387	46248.0	N/A
tiktoken	technical	6.291	1273319	0.2344	35843.8	N/A
tiktoken	web	4.509	1181076	0.0518	31697.6	N/A
tiktoken_cached	code	5.134	1219410	0.4309	4470460.0	1.0000
tiktoken_cached	english	4.985	1229895	0.1431	1456340.0	1.0000
tiktoken_cached	technical	6.376	1290529	0.2313	2472672.0	1.0000
tiktoken_cached	web	4.517	1183047	0.0518	536444.0	1.0000

Results and Discussion

Headline findings

Overall winner by mean MB/s: tiktoken at 5.253 MB/s and 0.2143 ms average latency.

Fastest code-domain tokenizer: tiktoken_cached at 5.134 MB/s.

Cached versus naive Python baseline: 1.365 to 0.788 MB/s, or 0.58x speedup, with peak memory rising from 207623.3 to 1235830.0 bytes.

Cold start ranking by mean latency favored naive (0.0965 ms) and tiktoken (0.0965 ms), while hf was slowest at 0.2733 ms.

Batch throughput favored the cached baseline at 1135351.3 docs/s, but tiktoken is absent from the batch table because the current adapter intentionally exposes single-document encoding only.

Exact-compatible caching preserved correctness at 1.0 exact match rate, and mean throughput for tiktoken_cached was 5.253 MB/s versus 5.167 MB/s for native tiktoken, but with far higher memory usage.

Discussion

The results support H1. Native tokenizer libraries clearly outperformed the Python baselines on MB/s and latency, with tiktoken and SentencePiece consistently ahead of naive Python code.

The results support H2. Ranking changed by domain, which matters because tokenizer selection for RAG preprocessing, web corpora, and code corpora should not rely on one aggregate benchmark number.

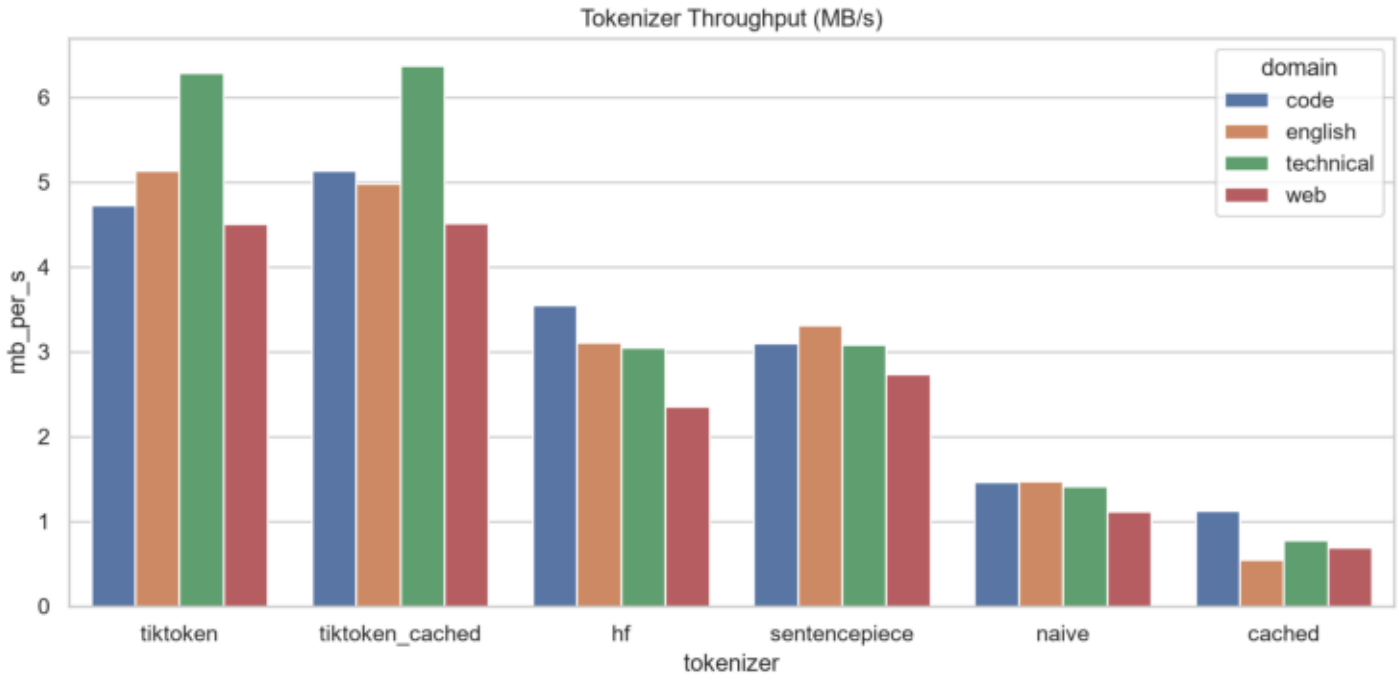
The results support H3 with caveats. Caching materially improved the Python baseline, but it increased memory and still depended on high substring reuse. Even after diversifying the synthetic corpus, the cache hit rate remained high, so the next step is to rerun on real corpora.

The results partially support H4. Batch throughput was much higher for adapters with encode_batch support, but batch results are not comparable for tiktoken until a batch-capable adapter path is added.

H5 is now supported empirically: the exact-compatible cached adapter matched tiktoken token IDs exactly across the benchmark corpus.

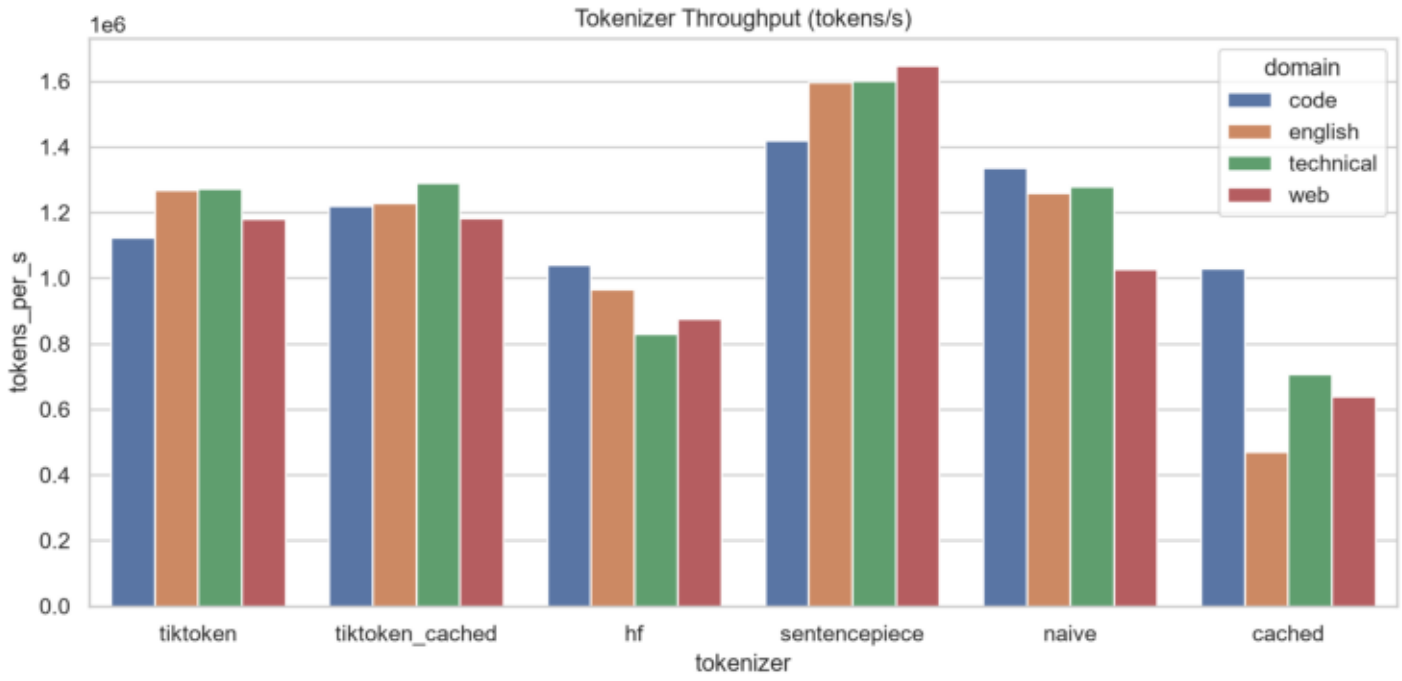
H6 is supported in a narrower sense: reducing Python overhead changed the result from a slowdown into a small speed win, but the memory cost remains large enough that the overall systems conclusion is still a tradeoff rather than a free improvement.

Throughput Mb S



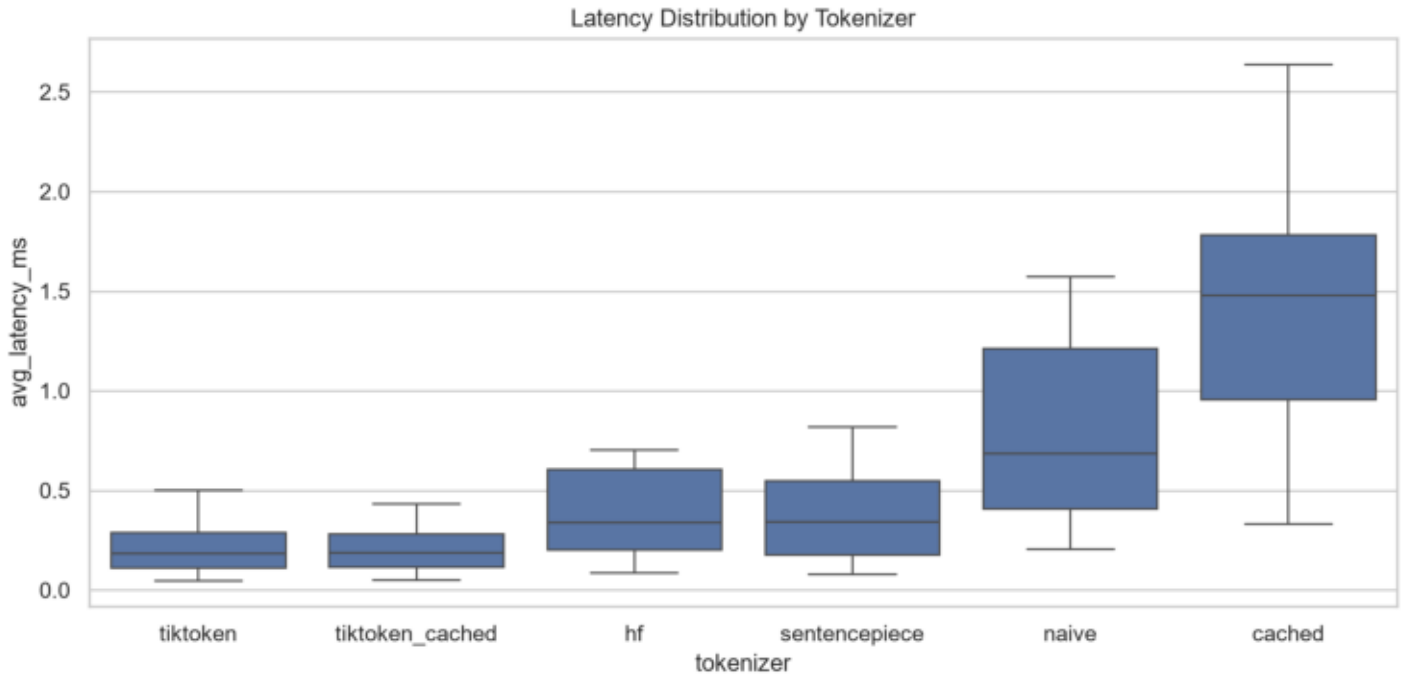
Experiment 1: Tokenizer throughput comparison in MB/s.

Throughput Tokens S



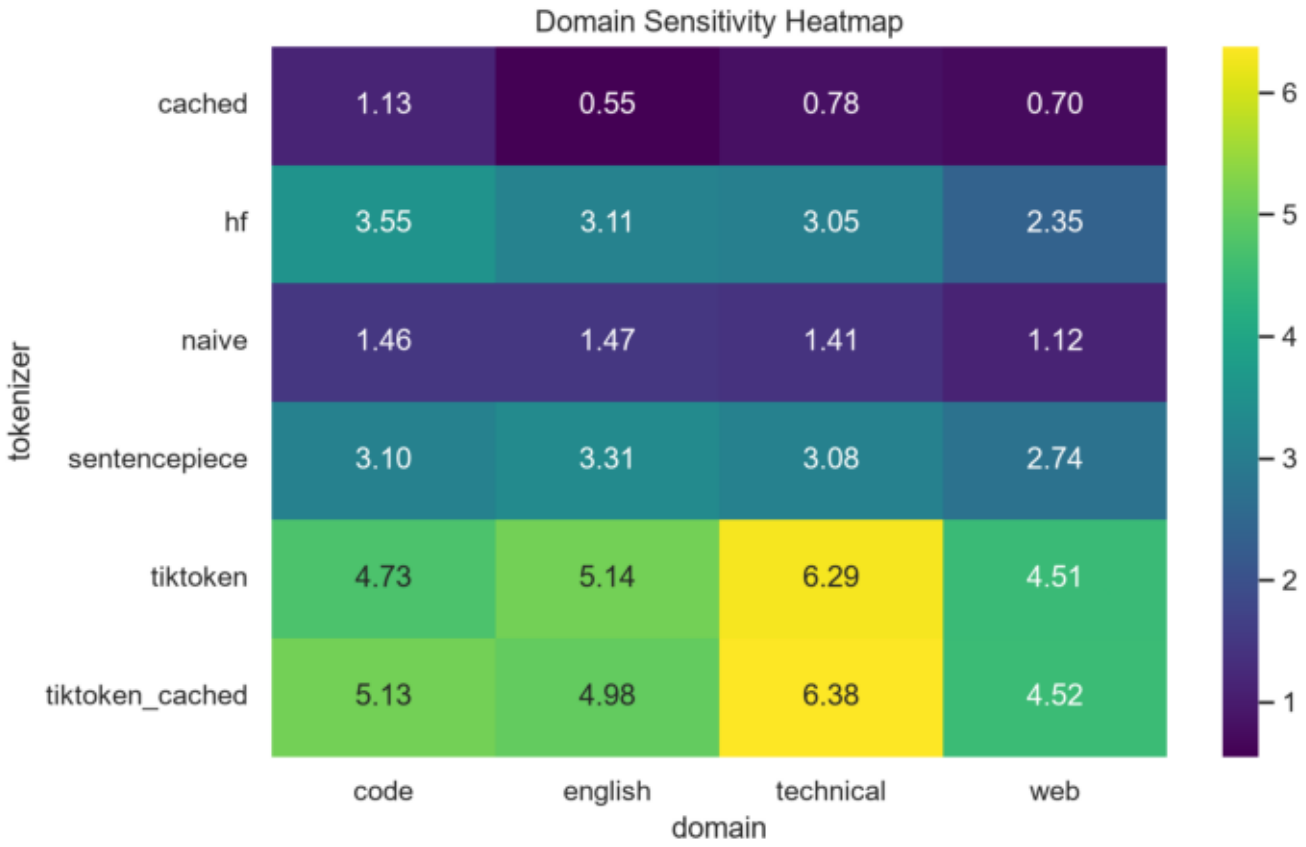
Experiment 1: Tokenizer throughput comparison in tokens/s.

Latency Distribution



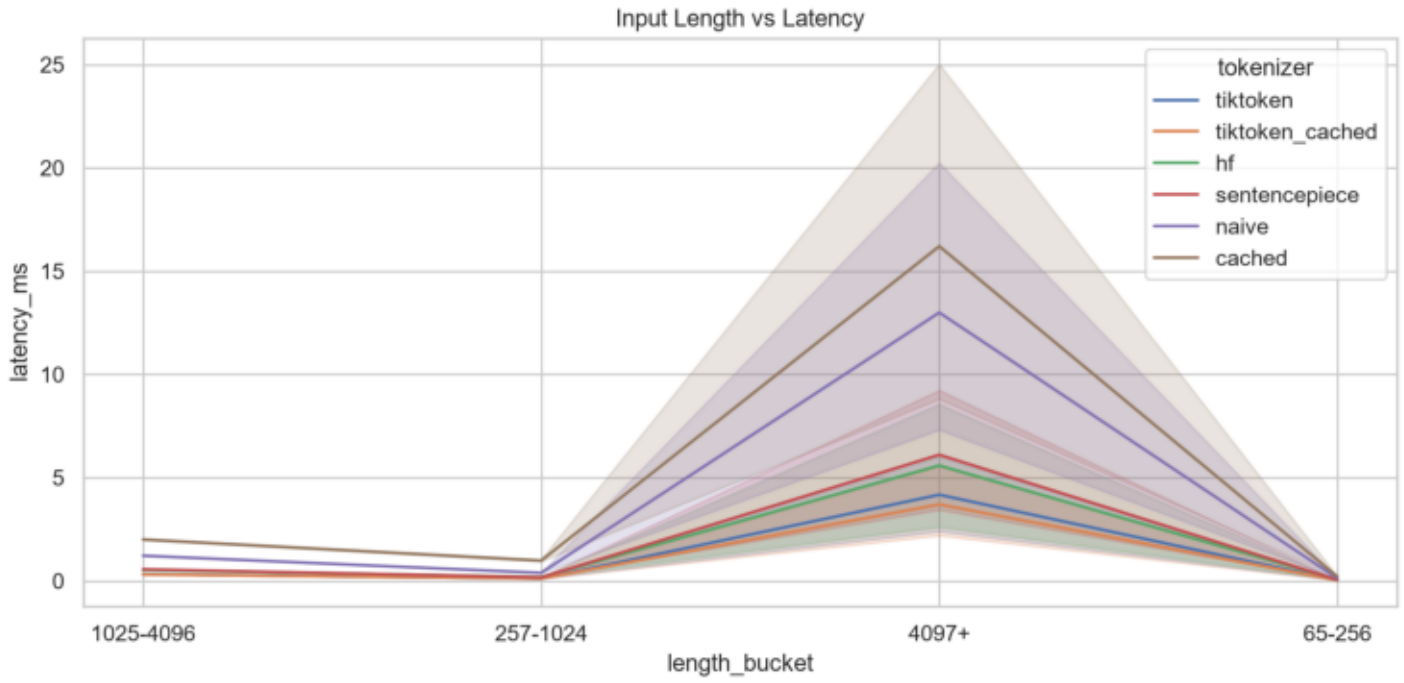
Latency distribution by tokenizer across benchmark trials.

Domain Heatmap



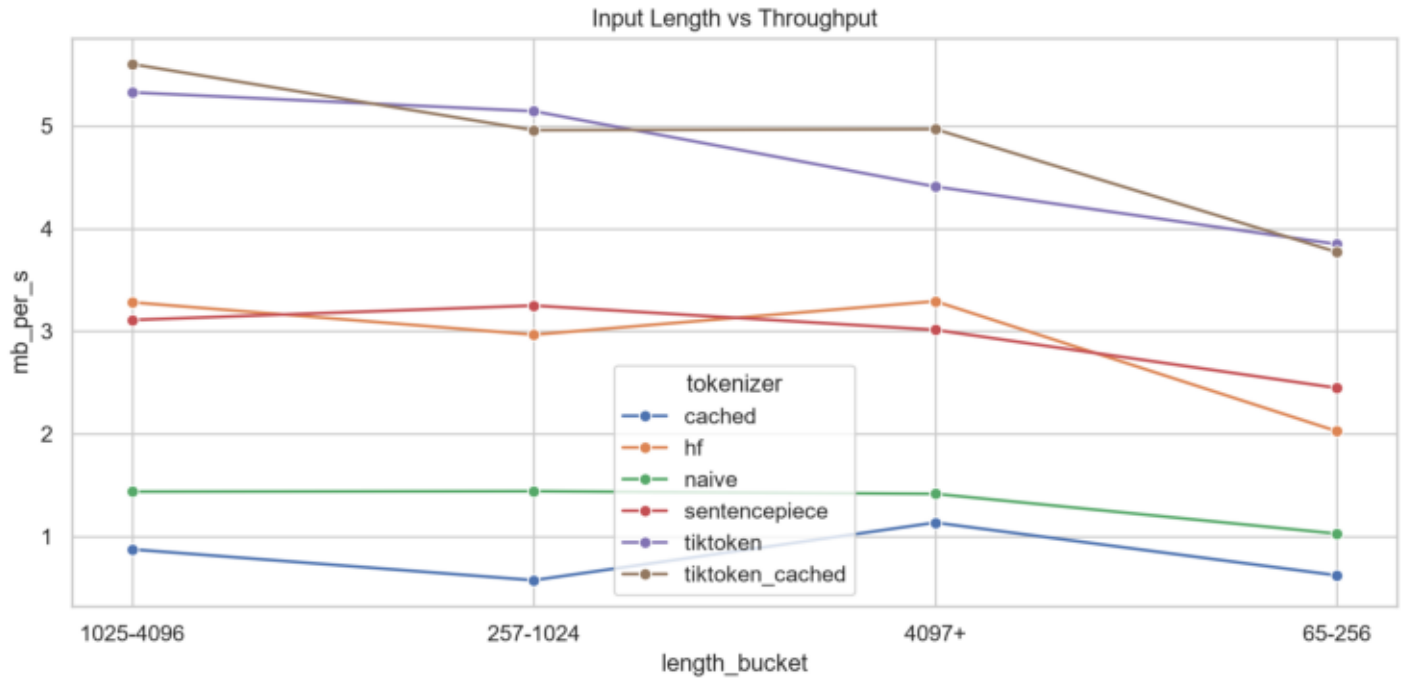
Experiment 2: domain sensitivity heatmap for throughput.

Length Vs Latency



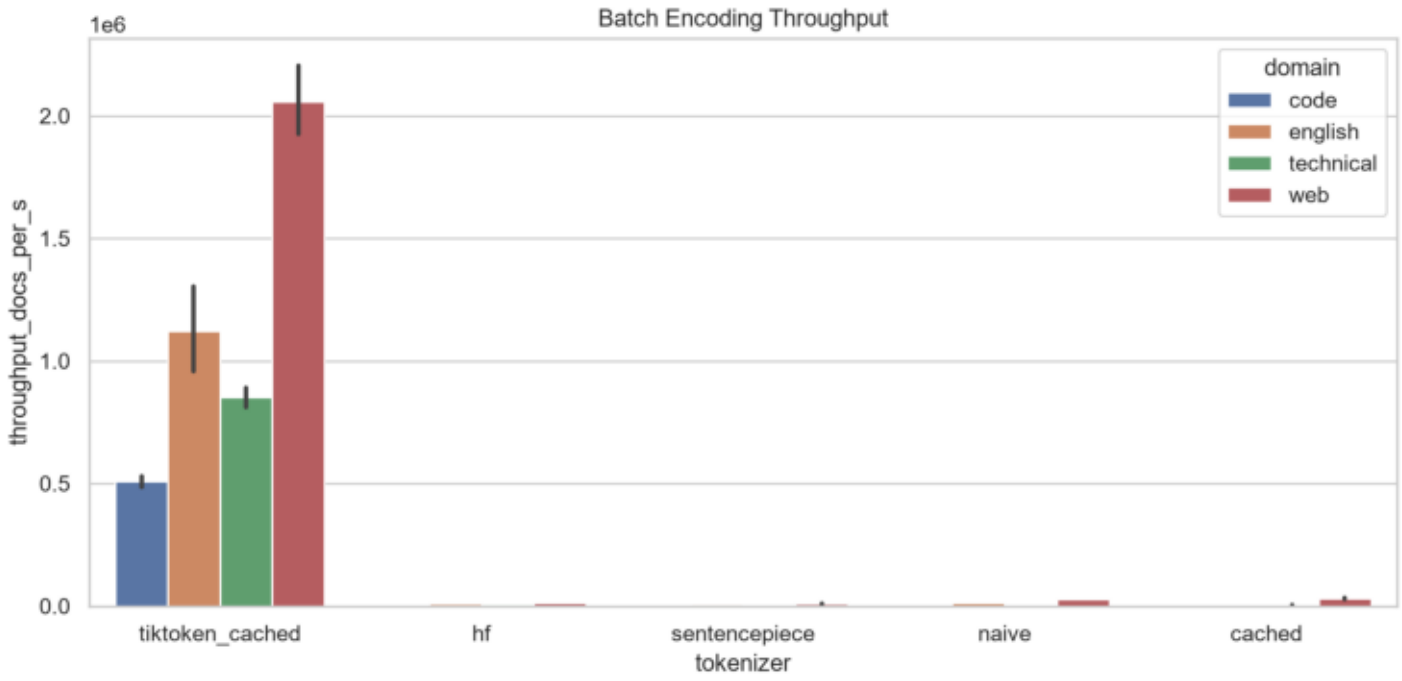
Experiment 3: input length versus latency.

Length Vs Throughput



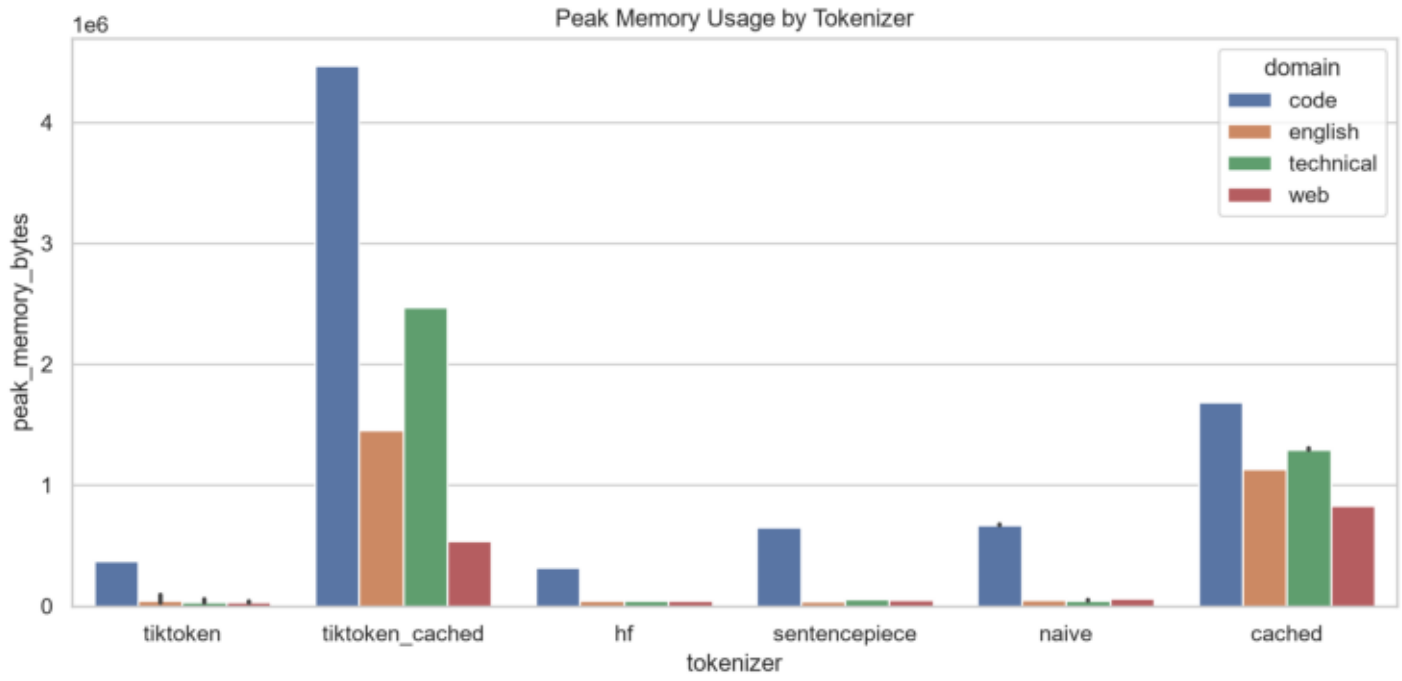
Experiment 3: input length versus throughput.

Batch Throughput



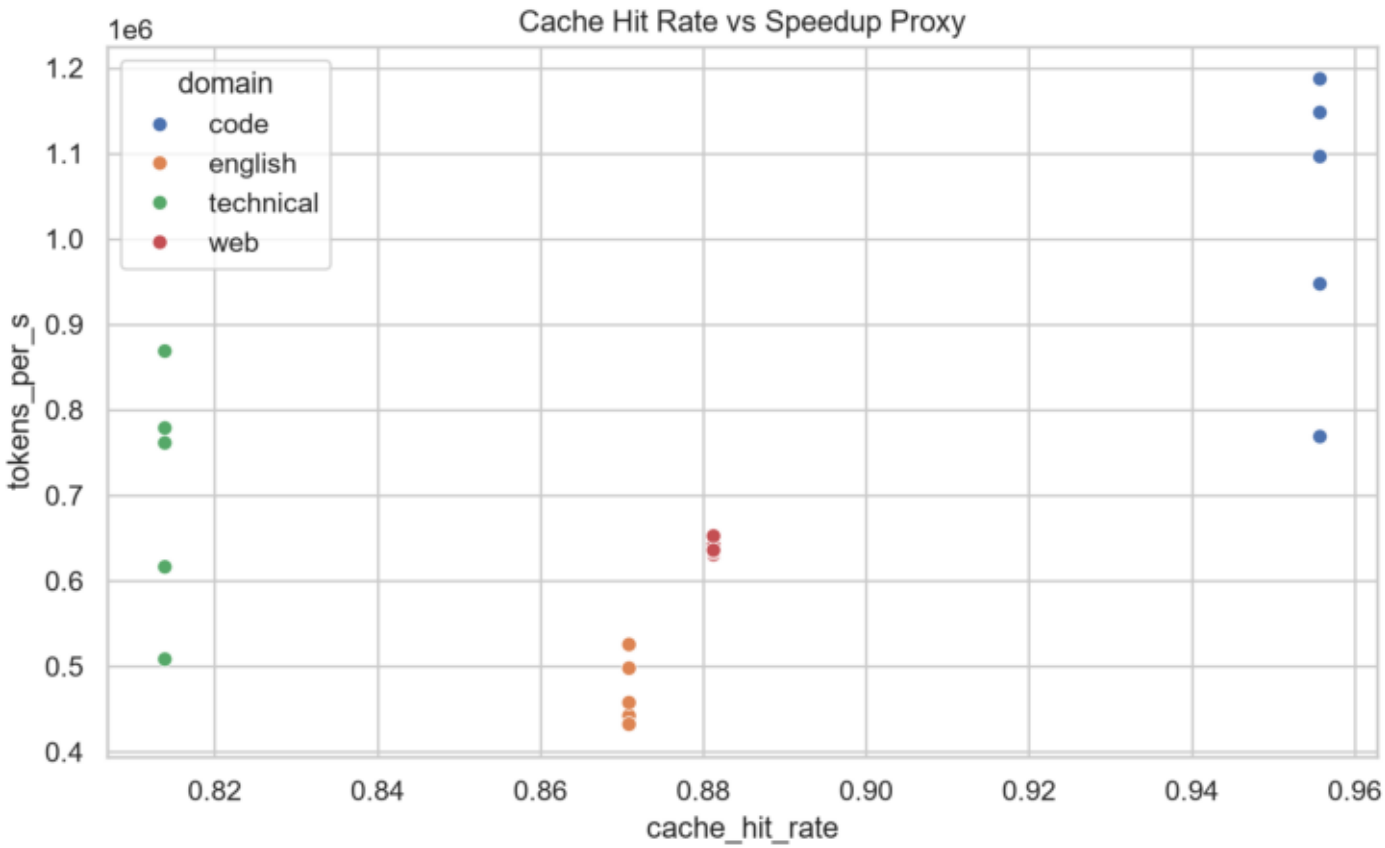
Experiment 4: batch encoding throughput.

Memory Usage



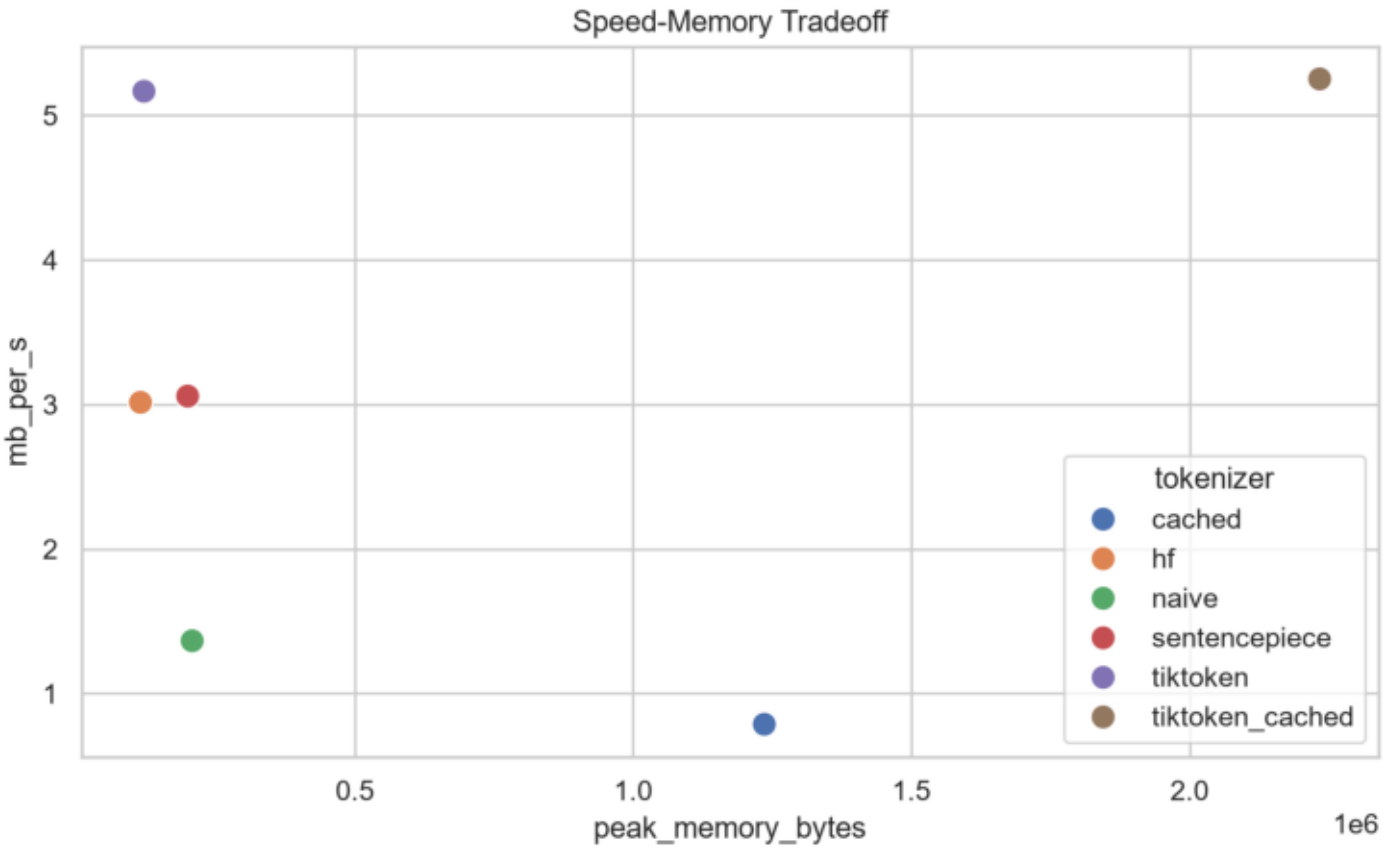
Memory usage by tokenizer and domain.

Cache Hit Rate Vs Speedup



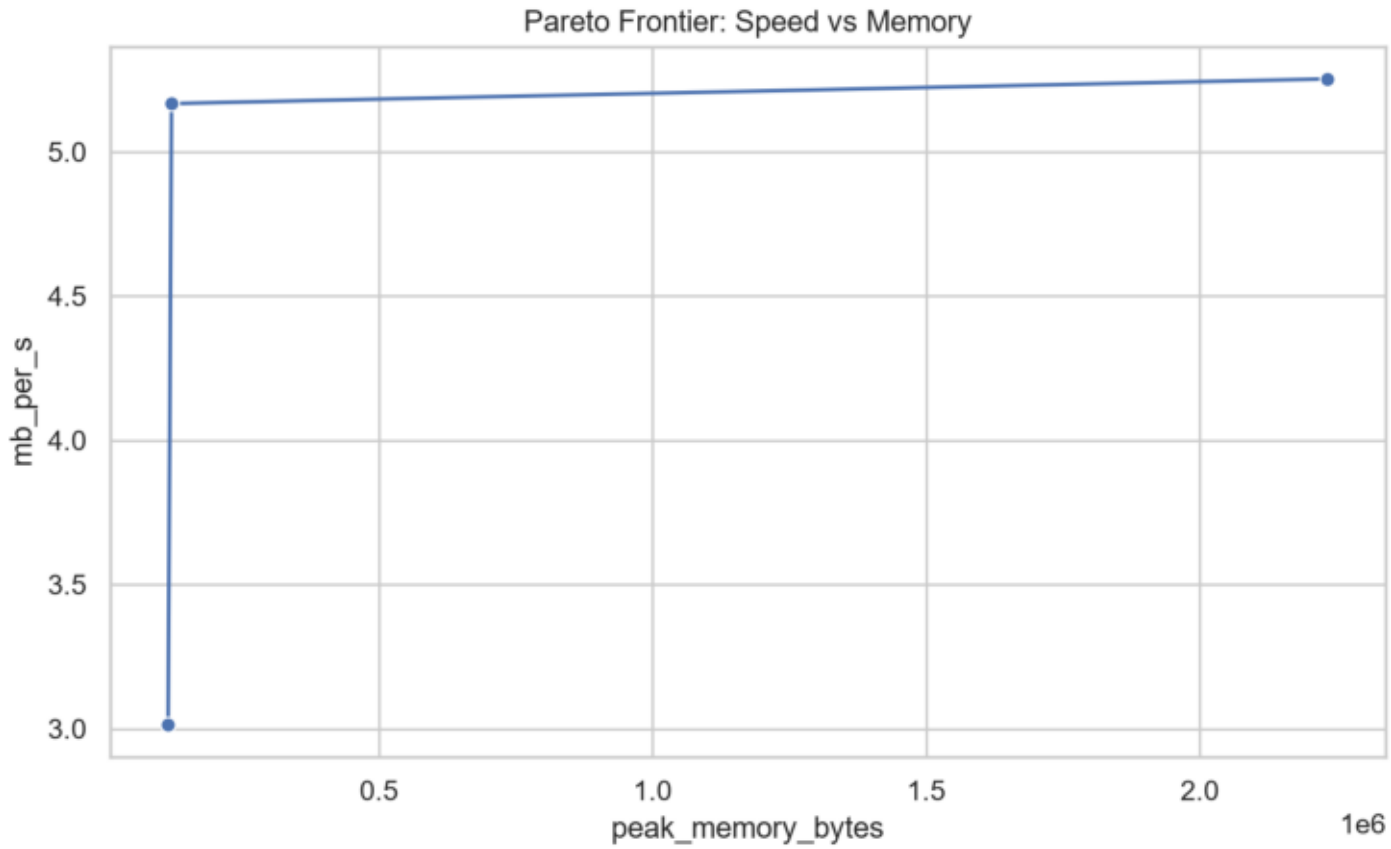
Experiment 6: cache hit rate versus throughput proxy.

Speed Memory Tradeoff



Speed-memory tradeoff across tokenizer implementations.

Pareto Frontier



Pareto frontier of throughput versus memory.